

APELLIDOS (en MAYÚSCULAS)

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

NOMBRE (en MAYÚSCULAS)

--	--	--	--	--	--	--	--	--	--	--	--

Cada hoja se entrega por separado y es obligatorio entregar todas las hojas. Se debe poner el **nombre completo en todas las hojas**. No se podrá entregar hasta pasados **20 minutos**. No se pueden usar hojas de sucio, para ello se dispone del espacio sobrante en cada hoja. El examen tendrá una duración máxima de **2:15 h**. No se podrá salir de los bucles usando **break o return**.

1.- (2 pts) Escribir una función que, pasándole como parámetros una matriz cuadrada de $N \times N$ caracteres (siendo N una cte ya definida) y una letra mayúscula, la modifique de forma que todas las posiciones fuera de las diagonales principales contengan el carácter ' ' (espacio) y éstas rellenas de la siguiente forma:

- si el carácter pasado como parámetro no es una letra mayúscula, ésta se generará aleatoriamente.
- la diagonal principal con letras mayúsculas crecientes empezando por la pasada como parámetro.
- la diagonal principal inversa con letras mayúsculas decrecientes empezando por la pasada como parámetro.
- en el caso de que N sea impar, el valor de la posición central de la matriz lo determina la diagonal principal.
- después de la Z se vuelve a utilizar la A en la diagonal principal en caso necesario, y después de la A se vuelve a utilizar la Z en la diagonal inversa en caso necesario.

Por ejemplo,

si N es 5 y la letra 'X', la matriz devuelta debería ser:

X				X
	Y		W	
		Z		
	U		A	
T				B

si N es 6, la matriz devuelta debería ser:

X					X
	Y			W	
		Z	V		
		U	A		
	T			B	
S					C

```
void rellenar (char mat[N][N], char letra)
{
    int i, j;
    char laux;
    if (letra < 'A' || letra > 'Z')
        letra = rand() % 26 + 'A';
    laux = letra;
    for (i=0; i<N; i++)
        for (j=0; j<N; j++)
            mat[i][j] = ' ';
    for (i=0, j=N-1; i<N && j>=0; i++, j--)
        {
            mat[i][j] = letra--;
            if (letra < 'A')
                letra = 'Z';
        }
    for (i=0, letra = laux; i<N; i++)
        {
            mat[i][i] = letra--;
            if (letra > 'Z')
                letra = 'A';
        }
}
```

2.- (2 ptos) Escribir un programa que lea de teclado un número entero n entre 3 y 10. A continuación se leerán caracteres por teclado hasta que hayan aparecido n veces dos caracteres iguales seguidos. Por ejemplo, si el número n es 4, una posible entrada de datos sería:

'j' 'j' 'F' '+' '5' 'g' 'g' '\$' 'a' 'm' 'T' 'V' 'V' 'h' '[' '#' 'k' '?' '?'

```
main()
{ unsigned n;
  char ant, act;

  do
  { printf("Introduzca el número de repeticiones a leer (3..10): ");
    scanf("%u", &n);
  } while (n<3 || n>10);

  printf("\n\nIntroduzca una secuencia de caracteres:\n");
  scanf("%c", &ant);

  do
  { scanf("%c", &act);
    if (ant == act)
      n--;
    ant=act;
  } while (n);
}
```

3.- (2 ptos) Escribir una función en C que, recibiendo como parámetro un número entero positivo, elimine de éste los dígitos que ocupan posiciones pares. Las unidades se considera que son la posición 0, las decenas la 1, y así sucesivamente. Por ejemplo, si el número pasado es 56821, lo devolverá como 62, si se le pasa 7 devolverá 0.

```
unsigned elim_pos (unsigned n)
{ unsigned aux = 0, pos;

  for (n/=10; n;/=100) //se obtiene el número invertido
    aux = aux*10 + n%10;

  for (n=aux%10; aux;/=10; ) //para darle la vuelta
    n = n*10 + aux%10;

  return (n);
}
```

APELLIDOS (en MAYÚSCULAS)

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

NOMBRE (en MAYÚSCULAS)

--	--	--	--	--	--	--	--	--	--	--	--	--

4.- (2 pts) Escribir un programa que, recibiendo como argumentos el nombre de un fichero de texto y una palabra, grabe en otro fichero, de nombre *salida.txt*, todas las líneas del primero después de quitarles todos los caracteres que aparecen antes de la primera aparición de la palabra pasada como parámetro. Las líneas en el fichero de salida deberán ir numeradas, empezando por la 1. Por ejemplo, si la palabra pasada es *“estadio”* y la primera línea leída del fichero es

“Los aficionados acudieron en gran número al estadio mucho antes del comienzo del partido.”

En el fichero de salida se escribirá la línea

“1 estadio mucho antes del comienzo del partido.”

```
int main (int argc, char *argv[])
{ FILE *f_in, *f_out;
  char línea[256], *pos;
  int num=1;

  if (argc<3)
  { printf ("Error en el número de parámetros.\n");
    return (1);
  }
  if (! (f_in = fopen(argv[1], "r")))
  { printf ("Error en la apertura del fichero %s.", argv[1]);
    return (2);
  }
  if (! (f_out = fopen("salida.txt", "w")))
  { printf ("Error en la apertura del fichero salida.txt.");
    fclose (f_in);
    return (3);
  }

  while (fgets(línea, 256, f_in))
  { if (pos=strstr(pos, argv[2]))
    strcpy(línea, pos);
    fprintf (f_out, "%d %s\n", num++, línea);
  }

  fclose(f_in);
  fclose(f_out);
  return(0);
}
```

5.- (2 ptos) Dada la estructura USUARIO definida como:

```
typedef struct
{ char nombre[25], apell[50], domic[60], cpost[6];
} USUARIO;
```

Escribir un programa en C que, dado el fichero de *USUARIOS* de nombre “*user.dat*”, guarde cada usuario del primero en un segundo fichero de nombre “*userx.dat*”, cambiando, para cada usuario, el bit que ocupa la posición correspondiente al primer dígito del código postal en todos los caracteres de los campos *nombre*, *apell* y *domic*. Por ejemplo, si el código postal de un usuario es “28137”, deberá cambiar el bit que ocupa la posición 2 (la de menor peso es la 0).

```
int main ()
{ FILE *f_in, *f_out;
  USUARIO user;
  char masc;
  int i;

  if (! (f_in = fopen("user.dat", "rb")))
  { printf ("Error en la apertura del fichero de entrada.");
    return (1);
  }
  if (! (f_out = fopen("userx.dat", "wb")))
  { printf ("Error en la apertura del fichero de salida.");
    fclose (f_in);
    return (2);
  }

  while (fread(&user, sizeof(USUARIO), 1, f_in))
  { masc = 1 << (user.cpost[0]-'0');
    for (i=0; user.nombre[i]; i++)
      user.nombre[i] ^= masc;
    for (i=0; user.apell[i]; i++)
      user.apell[i] ^= masc;
    for (i=0; user.domic[i]; i++)
      user.domic[i] ^= masc;
    fwrite(&user, sizeof(USUARIO), 1, f_out);
  }

  fclose (f_in);
  fclose(f_out);
  return (0);
}
```

APELLIDOS (en MAYÚSCULAS)

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

NOMBRE (en MAYÚSCULAS)

--	--	--	--	--	--	--	--	--	--	--	--	--

ALGUNOS PROTOTIPOS DE FUNCIONES DEFINIDAS EN C

*int fprintf(FILE *, const char *, ...);*

*int fscanf(FILE *, const char *, ...);*

*int fgetc(FILE *);*

*int fputc(int, FILE *);*

*char *fgets(char *, int, FILE *);*

*char *fputs(char *, FILE *);*

*unsigned fwrite(void *, unsigned, unsigned, FILE *);*

*unsigned fread(void *, unsigned, unsigned, FILE *);*

*char * strchr(const char *, int);*

*char * strstr(const char *, const char *);*

*char * strcat(char *, const char *);*

*int strcmp(const char *, const char *, unsigned);*

*int atoi(const char *);*